

Pruning Language Models for Low-End Hardware: An Exploration of Novel Structured and Unstructured Approaches

Fida Ullah, Mohammad Abdul Mannan Sami, Mahbubur Rahman Khan

Department of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh

Email: {fida.ullah, mannan.sami, mahbubur.khan}@northsouth.edu

Abstract—Deploying large language models (LLMs) like OPT-125m and OPT-1.3b on low-end hardware requires reducing computational complexity while maintaining performance. This study explores and evaluates six pruning strategies—neuron separation, Fourier transform, Wanda-based layer selection, residual correction, angle correction, and LoRA fine-tuning—applied to OPT-125m and OPT-1.3b. We assess perplexity on C4, WikiText-2, and PTB datasets at sparsity levels of 0%, 16.67%, 25%, and 33.33%. Each approach is detailed with algorithms, and we analyze trade-offs in inference perplexity. LoRA fine-tuning achieves the lowest perplexity at high sparsity, while Wanda-based pruning offers hardware-agnostic efficiency. Residual and angle corrections perform well at moderate sparsity. Resources are available at https://github.com/mahbubkousar/pruning_language_models.

Index Terms—Pruning, Language Models, Structured Pruning, Unstructured Pruning, Low-End Hardware

I. INTRODUCTION

Large Language Models (LLMs) such as OPT-125m and OPT-1.3b [1] have achieved impressive results in a variety of natural language processing tasks. However, their large size and heavy computational requirements make it difficult to deploy them on low-end hardware. Model pruning offers a promising solution by reducing model complexity while aiming to preserve performance. Structured pruning, in particular, can significantly speed up inference without requiring specialized hardware, compared to unstructured pruning [2].

In this project, we explore six different pruning methods—neuron separation, Fourier transform pruning, Wanda-based layer selection, residual correction, angle correction, and LoRA fine-tuning—applied to OPT-125m and OPT-1.3b. We evaluate these methods based on their ability to maintain low perplexity across three datasets (C4, WikiText-2, and PTB) under varying levels of sparsity (0%, 16.67%, 25%, and 33.33%). For each method, we describe the algorithm, report the results, and compare trade-offs between model size, performance, and computational efficiency.

II. BACKGROUND STUDY

Pruning LLMs reduces computational overhead by removing redundant parameters. Unstructured pruning, such as magnitude-based weight pruning [3], targets individual

weights but often requires specialized hardware for speedup. Structured pruning, like layer or neuron removal, is hardware-agnostic and optimizes inference speed [2]. The Wanda approach [4] selects layers for pruning based on weight-activation products, focusing on low-magnitude contributions. Corrective mechanisms, such as residual layers or LoRA fine-tuning [5], restore performance post-pruning. ArcFace loss [6], typically used in face recognition, enhances discriminative ability and is adapted here for language modeling. Prior work on OPT models [1] underscores the efficacy of combining pruning with corrective strategies, motivating our comprehensive evaluation.

III. METHODOLOGY

We prune OPT-125m and OPT-1.3b using six approaches, evaluating them on C4, WikiText-2, and PTB datasets. Each approach is described in a dedicated subsection, accompanied by its algorithm for clarity. The pruning pipeline is illustrated in Figure 1.

A. Neuron Subtraction

We first subtracted a neuron from the weight matrix such that it can represent approximately all the weights with small magnitudes to be zeroed out. We set the neuron as the average of the weights to be zeroed for each feature. If weights are not zeroed out it is equivalent to a matrix dot product.

Algorithm 1 Neuron Separation Pruning

- 1: **Input:** Weight matrix $W \in \mathbb{R}^{m \times n}$, data $X \in \mathbb{R}^{n \times k}$, neuron $N \in \mathbb{R}^{1 \times n}$, threshold τ
 - 2: Compute: $W' = W - N$
 - 3: Zero out weights in W' where $|W'| < \tau$
 - 4: Compute: $Y = W' \cdot X + N \cdot X$
 - 5: **Output:** Pruned output $Y \in \mathbb{R}^{m \times k}$
-

B. Fourier Transform

We found a fourier mechanism that is equivalent to a matrix dot product. We first transform the weight matrix into frequency domain by inverse fourier across the features. We then

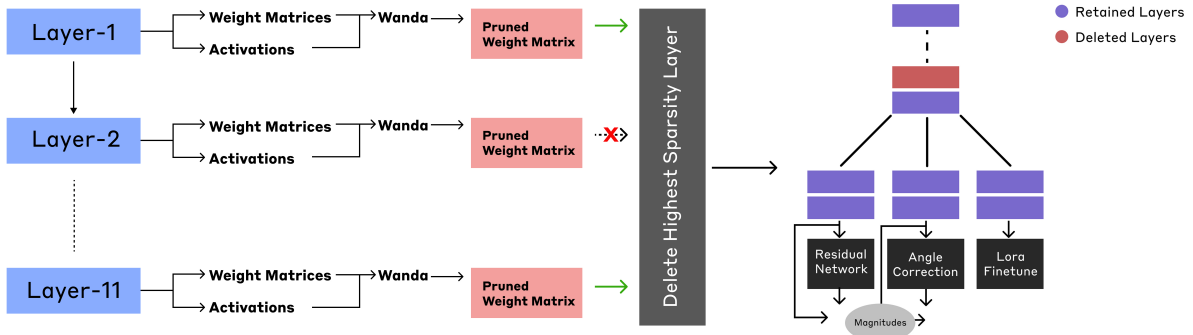


Fig. 1. Our proposed pruning pipeline for LLMs, including weight/layer pruning, corrective mechanisms, and optional LoRA fine-tuning.

do a standard dot product and fourier transform the result back to get the equivalent result. We hypothesized that since we transformed the weights to the frequency domain, we can get rid of the high frequency components since the low frequency components will be able to capture more global information. Unfortunately we found that approximation becomes very weak at zeroing out a few neurons and also the weight matrix now needed to hold both real and imaginary parts increases memory.

Algorithm 2 Fourier Transform Pruning

- 1: **Input:** Weight matrix $W \in \mathbb{R}^{m \times n}$, data $X \in \mathbb{R}^{1 \times n}$
 - 2: Transform: $W' = \text{FFT}^{-1}(W, \text{dim} = 0)$
 - 3: Zero out high-frequency components in W' (last neurons)
 - 4: Compute: $Y = \Re(\text{FFT}(\text{Complex}(X) \cdot W'.T))$
 - 5: **Output:** Pruned output $Y \in \mathbb{R}^{1 \times m}$
-

C. Wanda-Based Layer Selection

We decided to prune layers since structured pruning increases inference speed more than unstructured pruning and is hardware independent.

A layer selection criteria was needed for pruning the layers so we took the Wanda approach. We multiplied the activations with weights of fc1 and fc2 making almost two thirds of a layer. We then zeroed out the weights with magnitude less than a threshold. The layers with the least remaining weights were selected for pruning.

Algorithm 3 Wanda-Based Layer Selection

- 1: **Input:** Model layers $\{L_1, \dots, L_N\}$, activations A_l , threshold τ
 - 2: **for** each layer l **do**
 - 3: Compute: $S_l = W_l \cdot A_l$
 - 4: Zero out weights where $|S_l| < \tau$
 - 5: Compute fraction of remaining weights
 - 6: **end for**
 - 7: Select layers with lowest remaining weight fractions
 - 8: **Output:** Indices of layers to prune
-

We then tried to find out a corrective mechanism that can remove the effects of pruning. We decided to attach a residual layer at the end of the model to correct the hidden states from the last layer before the LM head. We put the residual layer at the end so that it is somewhat layer independent and can correct regardless of what layer is pruned above. This can have the advantage of being shareable like Lora adapters. The residual layer had the same structure and dimensions of feedforward layers of the model Transformer block.

D. Residual Correction

ArcFace loss was initially selected as the loss for training the residual layer because it increases the discriminative ability of the model. The transformed embedding will be more directed towards the target class in the LM head to compute logits. Because training the model with ArcFace Loss was not reducing the loss enough we added the standard cross entropy so that whatever the vectors produced will have angles somewhat closer to the target class. The extra FLOPs introduced was around 19.33GFLOPs which is less than an OPT transformer layer with around 30.89GFLOPs.

Algorithm 4 Residual Correction

- 1: **Input:** Hidden states $H \in \mathbb{R}^{b \times s \times d}$, ResidualModel $ResModel$, deleted layers D
 - 2: **for** each layer l **do**
 - 3: **if** $l \notin D$ **then**
 - 4: $H = \text{Layer}_l(H)$
 - 5: **end if**
 - 6: **end for**
 - 7: $H' = H + ResModel(H)$
 - 8: Compute logits: $L = \text{LMHead}(H')$
 - 9: **Output:** Corrected logits L
-

E. Angle Correction

We also tried another approach that uses only cross entropy, we first stored the magnitudes of the hidden state vectors from the last layer, then passed the hidden states through a model

with residual connection. We then converted the resultant hidden states to unit vectors multiplying the magnitude to restore their magnitude so that model only corrects the angle of the hidden states. The extra FLOPs introduced was around 19.33GFLOPs which is less than an OPT transformer layer with around 30.89GFLOPs.

Algorithm 5 Angle Correction

```

1: Input: Hidden states  $H \in \mathbb{R}^{b \times s \times d}$ , ResidualModel  $R$ ,
   deleted layers  $D$ 
2: Compute magnitudes:  $M = \|H\|$ 
3: for each layer  $l$  do
4:   if  $l \notin D$  then
5:      $H = \text{Layer}_l(H)$ 
6:   end if
7: end for
8:  $M = \|H\|$ 
9:  $H' = H + \text{Model}(H)$ 
10:  $H'' = \frac{H'}{\|H'\|}$ 
11:  $H''' = H'' \cdot M$ 
12: Compute logits:  $L = \text{LMHead}(H''')$ 
13: Output: Corrected logits  $L$ 

```

F. LoRA Fine-Tuning

This approach prunes layers using Wanda and fine-tunes with LoRA adapters to recover performance [5]. It minimizes computational overhead while adapting the model to target datasets, making it highly effective for low-end hardware.

Algorithm 6 LoRA Fine-Tuning

```

1: Input: Model, deleted layers  $D$ , LoRA adapters
2: for each layer  $l$  do
3:   if  $l \notin D$  then
4:     Process input through layer  $l$ 
5:   end if
6: end for
7: Fine-tune with LoRA adapters on target dataset
8: Compute logits via LM head
9: Output: Fine-tuned logits

```

IV. EXPERIMENTS

We evaluate OPT-125m and OPT-1.3b on C4, WikiText-2, and PTB datasets at sparsity levels of 0%, 16.67%, 25%, and 33.33%.

A. Experimental Setup

For residual and angle correction, we train on 500 C4 samples for 10 epochs, evaluating on 500 validation samples. LoRA fine-tuning is performed on WikiText-2 for pruned models. Layers are pruned based on Wanda selection (e.g., layers 3, 4, 7, 5, 6 for higher sparsity). Perplexity (PPL) is reported as the primary metric.

B. Results

Tables I, II, III, IV and V present perplexity results for angle correction and LoRA fine-tuning, as these approaches provided the most comprehensive data. Results for neuron separation, Fourier transform, and residual correction were incomplete due to poor performance or implementation challenges.

TABLE I
RESIDUAL ARC TRAINED ON WIKITEXT2 AND TESTED ON WIKITEXT2

Sparsity	Layers Deleted	Perplexity Wikitext2 Test Set
Base Model	Not Deleted	27.650
16.67%	3,4	38.248
25%	3,4,7	49.360
33.33%	3,4,7,5	94.001

TABLE II
ANGLE CORRECTOR TRAINED ON WIKITEXT2 AND TESTED ON WIKITEXT2

Sparsity	Not Wanda Pruning		Wanda Pruning	
	Layers Deleted	Perplexity	Layers Deleted	Perplexity
Base Model	Not Deleted	26.75	Not Deleted	26.75
16.67% - 5.56%	2,3	29.418	3,4	30.548
25% - 5.56%	3,4,7	39.462	3,4,7	39.462
33.33% - 5.56%	3,4,7,6	67.067	3,4,7,5	71.397

TABLE III
MODEL: FACEBOOK/OPT-125M FINETUNED ON C4 TRAIN SET AND TESTED ON C4 TEST SET

Sparsity	Perplexity C4 Validation Set
Base Model	27.1
16.67%	30.579
25%	35.538
33.33%	43.955

TABLE IV
MODEL: FACEBOOK/OPT-125M FINETUNED ON WIKITEXT2 AND TESTED ON WIKITEXT2 AND PTB

Sparsity	Ptb Perplexity	Wikitext2 Perplexity	Fine Tuned on Wikitext2
Base Model	35.325	27.64	No
16.67%	45.137	20.882	Yes
25%	54.190	23.519	Yes
33.33%	64.108	28.860	Yes

TABLE V
MODEL: FACEBOOK/OPT-1.3B FINETUNED ON WIKITEXT2 AND TESTED ON WIKITEXT2

Sparsity	Ptb Perplexity	Wikitext2 Perplexity	Fine Tuned on Wikitext2
Base Model	17.813	14.624	No
16.67%	26.960	10.910	Yes
25%	59.598	16.152	Yes
33.33%	87.738	26.894	Yes



Fig. 2. Perplexity of pruning methods on WikiText-2 dataset for OPT-125m and OPT-1.3b models. LoRA fine-tuning (OPT-1.3b) achieves the lowest perplexity, especially at 16.67% sparsity. Wanda pruning refers to layer selection based on weight-activation products.

V. COMPARATIVE ANALYSIS

We compare the six pruning approaches based on perplexity, computational efficiency, and suitability for low-end hardware:

- **Neuron Subtraction:** This unstructured method introduces significant approximation errors, leading to high perplexity and increased computational overhead making it impractical for low-end hardware.
- **Fourier Transform:** By operating in the frequency domain, this approach doubles memory requirements and yields weak approximations at higher sparsity levels.
- **Wanda-Based Layer Selection:** This structured pruning method efficiently identifies layers for removal based on weight-activation products. When paired with corrective methods or LoRA, it achieves competitive perplexity (e.g., 30.548 PPL for OPT-125m at 16.67% sparsity with angle correction, Table II). Its hardware-agnostic nature makes it a robust foundation for pruning.
- **Residual Correction:** The residual layer (19.33 GFLOPs) enhances hidden state correction at moderate sparsity levels (16.67%–25%) trained with summation of cross entropy and arcfuse loss, but its performance degrades at 33.33% sparsity due to limited discriminative capacity.
- **Angle Correction:** By preserving magnitudes and adjusting angles, this method (19.33 GFLOPs) performs well at low to moderate sparsity. For OPT-125m, it achieves 29.418 PPL at 16.67% sparsity without Wanda and 30.548 PPL with Wanda (Table II) and does better than residual correction. However, at 33.33% sparsity, perplexity rises sharply (e.g., 71.397 PPL with Wanda), limiting its effectiveness at higher sparsity.
- **LoRA Fine-Tuning:** This approach consistently delivers the lowest perplexity across all sparsity levels for both models. For OPT-125m, it achieves 20.882 PPL at 16.67% sparsity and 28.860 PPL at 33.33% sparsity (Table IV). For OPT-1.3b, it achieves an impressive 10.910 PPL at 16.67% sparsity and 26.894 PPL at 33.33% sparsity (Table V). Its low computational overhead and

adaptability make it ideal for low-end hardware. However specific weights are learned for each sparsity lacking the generalized corrective effect of Angle Correction and Residual Correction.

LoRA fine-tuning stands out due to its ability to recover performance with minimal computational cost, particularly for OPT-1.3b at 16.67% sparsity (10.910 PPL). Wanda-based pruning provides a versatile foundation for layer selection, while angle correction is effective at moderate sparsity (e.g., 29.418 PPL for OPT-125m at 16.67%). Residual correction shows promise but lacks comprehensive results. Neuron separation and Fourier transform are less viable due to high computational and memory demands, making them unsuitable for low-end hardware.

VI. FUTURE WORK

There are several directions we would like to explore based on our current findings. One idea is to improve the Fourier-based pruning method by pruning weights in the frequency domain and then removing the imaginary parts of the pruned matrices to reduce memory usage. After this step, we would fine-tune the model to recover any performance loss. Another idea is to design models that take a layer’s weights from a teacher model as input and generate the corresponding student model weights automatically. This could make pruning more efficient. We also plan to try using transformer blocks instead of simple feed-forward layers for the residual and angle correction models. While this would increase computational cost, it could help the corrective models become more expressive and improve final model performance. Overall, these improvements aim to achieve better sparsity-performance trade-offs for running large models on low-end hardware.

VII. CONCLUSION

In this project, we evaluated six pruning methods for reducing the size and computational cost of large language models, focusing on OPT-125m and OPT-1.3b. Our results show that LoRA fine-tuning consistently achieved the best performance, maintaining the lowest perplexity even at higher sparsity levels. Wanda-based layer selection also proved to be an effective and hardware-friendly approach, especially when combined with corrective techniques like residual or angle correction. Residual and angle correction methods performed well at moderate sparsity but struggled at higher sparsity rates.

On the other hand, neuron separation and Fourier transform pruning were less effective due to higher computational overhead and weaker results, making them less suitable for low-end hardware. Based on our findings, future work could focus on improving the Fourier-based approach, exploring transformer-based residual correction models, and developing teacher-student frameworks for more efficient pruning. Overall, our project highlights the potential of combining structured pruning and corrective methods to make LLMs more accessible for resource-constrained environments.

REFERENCES

- [1] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [2] T. Hoeffer, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” *arXiv preprint arXiv:2102.00554*, 2021.
- [3] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” *arXiv preprint arXiv:1510.00149*, 2015.
- [4] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” *arXiv preprint arXiv:2306.11695*, 2023.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [6] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “Arcface: Additive angular margin loss for deep face recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 5962–5978, 2019.